UNIVERSITY OF CENTRAL PUNJAB

Faculty of Information Technology

Cyber Security

Week 3 — Vulnerabilities & CVE

What Vulnerabilities Are • Common Weaknesses • OWASP Top 10
CVE, CVSS & NVD • Scanning Tools • Safe Practice Labs

Instructor: Hafiz Usama Ishtiaq

BSCS • 3 Credit Hours • Fall 2026

QUICK RECAP

Where We Are So Far

Week 1	Cybersecurity basics, the CIA Triad, and key terminology.
Week 2	Threats and threat actors, internal vs external, attack vectors and common attacks.
A weakness = a vulnerability	Remember: attackers use an EXPLOIT to take advantage of a VULNERABILITY.
This week	We focus on the vulnerabilities themselves — the weaknesses attackers look for.

WEEK 3 ROADMAP

What We Will Cover This Week

1

Vulnerabilities Explained

What a vulnerability is, and where weaknesses come from

2

Common System Vulnerabilities

The most frequent weaknesses in real systems

3

CVE, CVSS & NVD

How the world names and scores known vulnerabilities

4

Tools & Safe Practice

Scanners and legal lab environments for hands-on learning

DEFINITION

What is a Vulnerability?

A vulnerability is a weakness or flaw in a system that an attacker can use to break security.

Simple analogy: a vulnerability is like an unlocked window in a house. The window itself does no harm but it gives a burglar a way in.

Vulnerability

The weakness (unlocked window)

Threat

The danger (a burglar exists)

Exploit

The method used (climbing through the window)

ROOT CAUSES

Where Do Vulnerabilities Come From?

Coding mistakes

Bugs and errors left in software during development.

Poor configuration

Systems set up carelessly — default passwords, open ports.

Outdated software

Old programs missing the latest security patches.

Weak design

Security not planned from the start of the project.

Human error

Users tricked, or admins who misconfigure settings.

Third-party parts

A weakness inside a library or plugin you rely on.

THE BIG PICTURE

Categories of Vulnerabilities

Weaknesses can appear in four broad areas of any system:

Software

Bugs in code, unpatched apps, insecure web pages

Hardware

Old devices, missing firmware updates, physical access

Network

Open ports, weak Wi-Fi, unencrypted connections

Human

Weak passwords, falling for phishing, poor habits

Common System Vulnerabilities (1 of 3)

Weak passwords	**Short, simple or reused passwords that are easy to guess.** **Example:** Using "password123" or the same password on every site.
Missing updates	**Known bugs left unpatched because software isn't updated.** **Example:** Running an old Windows version with a known security hole.
Default settings	**Devices left with the factory username and password.** **Example:** A router still using admin / admin after installation.
Open ports/services	**Unused network services left switched on and exposed.** **Example:** A database reachable from the internet with no protection.

Common System Vulnerabilities (2 of 3)

No encryption	**Data sent or stored as plain text anyone can read.** **Example:** Login details sent over HTTP instead of HTTPS.
Poor input checks	**Web forms that don't clean what users type in.** **Example:** A search box that lets an attacker inject database commands.
Excess privileges	**Users given more access rights than they actually need.** **Example:** A normal user account that can also change system settings.
Misconfiguration	**Security features turned off or set up incorrectly.** **Example:** A cloud storage bucket accidentally left open to the public.

Common System Vulnerabilities (3 of 3)

Unvalidated files	**Allowing risky file uploads without checking them.** **Example:** A site that lets a user upload a script disguised as an image.
Weak session control	**Login sessions that can be stolen or reused.** **Example:** A session ID that never expires, letting an attacker reuse it.
No logging/monitoring	**Attacks go unnoticed because nothing is recorded.** **Example:** A breach discovered months later because no logs were kept.
Unpatched plugins	**Add-ons and libraries with known holes.** **Example:** An outdated WordPress plugin that lets attackers take over.

The OWASP Top 10

OWASP (Open Worldwide Application Security Project) publishes the "Top 10" — the ten most critical web application security risks.

Why it matters: it is a free, world-recognized checklist. Developers and students use it to learn what to test for and how to build safer web apps.

1. **Broken Access Control**
2. **Cryptographic Failures**
3. **Injection**
4. **Insecure Design**
5. **Security Misconfiguration**
6. **Vulnerable Components**
7. **Authentication Failures**
8. **Data Integrity Failures**
9. **Logging & Monitoring Failures**
10. **Server-Side Request Forgery**

Three You Must Know

SQL Injection (SQLi)

Inserting database commands through a web input field.

Example: Typing ' OR '1'='1 into a login box to bypass the password.

Cross-Site Scripting (XSS)

Injecting malicious scripts that run in other users' browsers.

Example: A comment that steals the cookies of everyone who views it.

Broken Access Control

Users reaching data or actions they should not be allowed to.

Example: Changing the URL id from user=5 to user=6 to see another account.

SPECIAL CASE

Zero-Day Vulnerabilities

A zero-day is a vulnerability that is unknown to the software maker — so there is no patch yet, and defenders have "zero days" to prepare.

Very dangerous

No fix exists yet, so normal defenses may not stop it.

Highly valued

Attackers and even governments pay a lot for zero-days.

Race against time

Once found, vendors rush to release an emergency patch.

Example: an attacker exploits a brand-new browser flaw before anyone knows it exists.

What is a CVE?

CVE = Common Vulnerabilities and Exposures. It is a public list that gives every known vulnerability a unique ID, so everyone refers to it the same way.

Why it's useful: instead of everyone using different names for the same bug, a CVE ID gives one shared name — like an ID card for each vulnerability.

Run by: MITRE, a non-profit, supported by the global security community.

A real example

CVE-2021-44228 — "Log4Shell"

A critical flaw in the popular Log4j logging library that let attackers run code on millions of servers worldwide in 2021.

HOW TO READ IT

Anatomy of a CVE ID

CVE

The prefix — always "CVE"

-

2021

The year it was reported

-

44228

A unique number for that year

So CVE-2021-44228 means:

"The vulnerability numbered 44228, reported in the year 2021." Anyone, anywhere can look up this exact ID and know precisely which flaw is meant.

SCORING SEVERITY

CVSS — How Serious Is It?

CVSS (Common Vulnerability Scoring System) rates each vulnerability from 0.0 to 10.0 — higher means more dangerous.

0.0	0.1 – 3.9	4.0 – 6.9	7.0 – 8.9	9.0 – 10.0
None	Low	Medium	High	Critical

The score is based on things like how easy the attack is, and how much damage it can do.

Example: Log4Shell (CVE-2021-44228) scored 10.0 — Critical — because it was easy to exploit and gave attackers full control.

WHERE TO LOOK THEM UP

Vulnerability Databases

NVD

The U.S. National Vulnerability Database — adds CVSS scores and details to every CVE.

Example: Search nvd.nist.gov for a CVE to see its severity and fixes.

MITRE CVE List

The official master list where CVE IDs are created and stored.

Example: cve.org / cve.mitre.org — the source of the CVE ID itself.

Vendor advisories

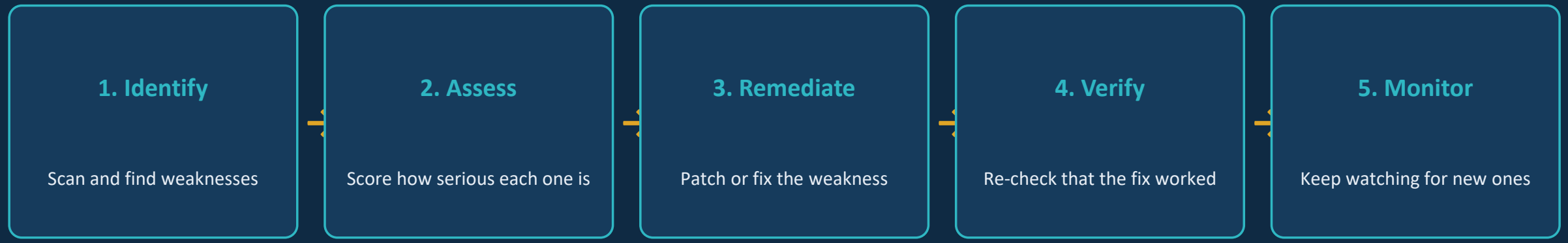
Security notices from companies (Microsoft, Cisco, etc.).

Example: "Patch Tuesday" — Microsoft's monthly batch of security fixes.

THE PROCESS

Vulnerability Management Lifecycle

Organizations follow a repeating cycle to stay protected:



Because new vulnerabilities appear every day, this cycle never really stops.

TOOLS · SCANNING

Vulnerability Scanning Tools

These tools automatically search a system for known weaknesses. (Names shown for learning.)

Nessus / OpenVAS

Scan a computer or network and list its known vulnerabilities with severity.

Nmap

Discovers devices, open ports and running services on a network.

OWASP ZAP

A free web-app scanner that finds issues like XSS and injection.

Burp Suite (Community)

Popular tool for testing and analyzing web application security.

Nikto

A simple scanner for common web server problems.

Wireshark

Captures and inspects network traffic to spot insecure data.

TOOLS · PRACTICE

Safe Places to Practice

Practice ONLY on systems you own or that are made for training. Never test systems without permission.

TryHackMe

Beginner-friendly guided cybersecurity lessons in your browser.

Hack The Box

Hands-on labs and challenge machines for practising skills.

OWASP Juice Shop

A deliberately vulnerable web app made for safe learning.

DVWA / bWAPP

"Damn Vulnerable Web App" — practise web attacks legally at home.

Metasploitable

A vulnerable virtual machine to practise scanning safely.

Kali Linux

A free OS pre-loaded with security tools for learning.

IMPORTANT

Use Your Skills Legally & Ethically

Testing or attacking a system without clear permission is ILLEGAL — even if you mean no harm. It can lead to fines, expulsion, or jail.

Get permission

Only test systems you own or are authorized to test.

Use lab environments

Practise on Juice Shop, DVWA, TryHackMe — built for this.

Report responsibly

If you find a real bug, tell the owner privately, don't exploit it.

Respect privacy

Never access or share other people's data.

PRACTICE ACTIVITY 1

Look Up a Real CVE

A simple, safe activity you can do right now in a browser — no installation needed.

Step 1	Go to nvd.nist.gov (the National Vulnerability Database).
Step 2	Search for CVE-2021-44228 (Log4Shell).
Step 3	Read its description — what software is affected?
Step 4	Find its CVSS score and severity (it is 10.0, Critical).
Step 5	Note the recommended fix / patched version.

PRACTICE ACTIVITY 2

Explore OWASP Juice Shop

OWASP Juice Shop is a safe, deliberately vulnerable web app — perfect for first practice.

Step 1	Open the free demo at owasp.org/www-project-juice-shop (or run it locally).
Step 2	Explore the shop like a normal user first — browse products, make an account.
Step 3	Try the built-in "Score Board" of beginner challenges.
Step 4	Spot how user input is handled — this is where many bugs live.
Step 5	Match what you find to the OWASP Top 10 categories.

DEFENSE

How to Reduce Vulnerabilities

Patch regularly

Install updates quickly to close known holes.

Strong config

Change default passwords; turn off unused services.

Least privilege

Give users and apps only the access they need.

Validate input

Clean everything users type to block injection.

Encrypt data

Use HTTPS and encryption for data in transit and storage.

Scan & monitor

Run scanners and keep logs to catch issues early.

QUICK REFERENCE

Key Terms at a Glance

Vulnerability

A weakness in a system.

Exploit

Code/method that uses a weakness.

Zero-day

A flaw with no patch yet.

CVE

A unique ID for a known vulnerability.

CVSS

A 0–10 severity score.

NVD

Database with CVE details & scores.

OWASP Top 10

Top web application risks.

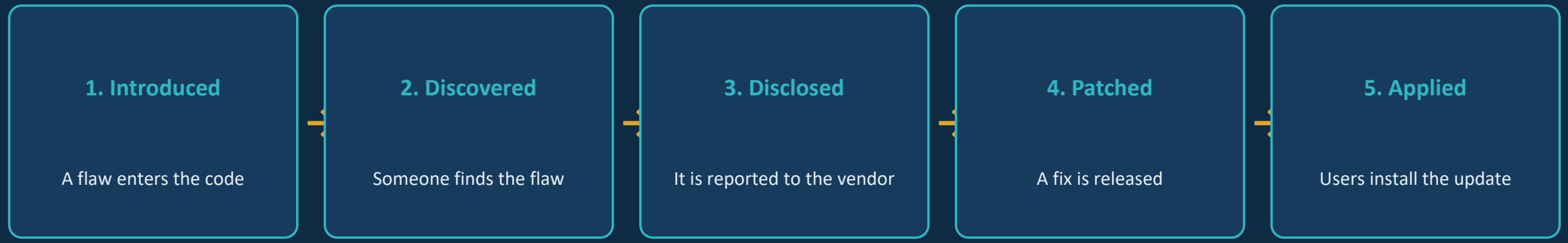
Patch

An update that fixes a weakness.

FROM DISCOVERY TO FIX

The Life of a Vulnerability

Every vulnerability travels through these stages — the danger is highest before a patch exists:



The danger window

Between DISCOVERED and PATCHED, attackers may already know the flaw while users are still exposed. This is why fast patching matters so much.

REAL-WORLD CASE STUDY

When a Vulnerability Isn't Patched

Equifax data breach (2017): attackers used a KNOWN, unpatched vulnerability to steal the personal data of ~147 million people.

The weakness	**A known flaw in Apache Struts (a web framework) with a fix already available.** **Example:** CVE-2017-5638 — a patch existed months before the attack.
What went wrong	**The patch was never applied, so the door stayed open.** **Example:** No proper patch process meant the fix was missed.
The lesson	**Knowing about a vulnerability isn't enough — you must actually fix it.** **Example:** Fast, tracked patching would have prevented the breach.

Responsible Disclosure & Bug Bounties

Responsible disclosure means reporting a vulnerability privately to the owner so they can fix it before attackers abuse it.

Find it legally

Discover a bug in a program that allows testing.

Report privately

Tell the owner directly — don't post it publicly or exploit it.

Give them time

Let the vendor create and release a fix.

Earn rewards

Many companies pay "bug bounties" for valid reports.

Platforms like HackerOne and Bugcrowd connect ethical hackers with companies that welcome (and reward) bug reports.

RECAP

Week 3 Summary

- A vulnerability is a weakness attackers exploit — like an unlocked window in a house.
- They come from coding mistakes, poor configuration, outdated software, weak design and human error.
- Common weaknesses: weak passwords, missing updates, default settings, no encryption, poor input checks.
- The OWASP Top 10 lists the most critical web risks (injection, XSS, broken access control, and more).
- CVE gives each known vulnerability a unique ID; CVSS scores its severity 0–10; NVD stores the details.
- Practice safely and legally using labs like Juice Shop, DVWA and TryHackMe — never test without permission.

CHECK YOUR UNDERSTANDING

Review Questions

1. Define a vulnerability. How is it different from a threat and an exploit?
2. List four common system vulnerabilities and give an example of each.
3. What is the OWASP Top 10, and why is it useful for students and developers?
4. What does a CVE ID tell you? Explain each part of CVE-2021-44228.
5. What is CVSS, and what do scores of 3.0, 6.5 and 9.8 mean?
6. Name two safe, legal ways to practise finding vulnerabilities.

WEEK 4 PREVIEW

Coming Next: Common Cyber Threats & Service Attacks

- Common cyber threats in depth
- Denial-of-service and service attacks
- Footprinting and reconnaissance basics